DEPARTMENT EVENT TICKET BOOKING SYSTEM

School of Computing

Bachelor of Technology
in
Computer Science & Engineering

By

SANKEPALLE PRAVALLIKA (VTU25243)

10211CS224
Full Stack Application Development
Slot : E2-B20

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled ”Department Event Ticket Booking System” by Sankepalle Pravallika (VTU25243) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr. Hemamalini.S

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Coordinator

Dr. Sumithra.M

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(SANKEPALLE PRAVALLIKA)

Date:06/05/2026

ABSTRACT

The Department Event Ticket Booking System is a full-stack web application designed to automate and simplify the process of managing and booking departmental events. Developed using React.js for the frontend and Node.js with Express.js for the backend, the system provides a responsive and user-friendly interface for students, along with an efficient admin dashboard for event management. It allows users to register and log in securely, browse available events, book tickets, and receive instant confirmation. Advanced features such as OTP-based authentication, QR code ticket generation, email notifications, and optional payment integration enhance security and user experience. The system uses a MySQL database to ensure reliable data storage and maintain relationships between users, events, and bookings. Overall, the application demonstrates the implementation and modern web development practices to deliver a seamless and efficient event booking solution.

Keywords: Full-Stack Development, React.js, Node.js, MySQL, Event Booking System, REST API,Dynamic, Web Application.

LIST OF FIGURES

LIST OF TABLES

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DOM	Document Object Model
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
REST	Representational State Transfer
SPA	Single Page Application
SQL	Structured Query Language

TABLE OF CONTENTS

Chapter 1

INTRODUCTION

1.1 Introduction

The rapid digitalization of services has transformed how people interact with events, shifting from manual ticketing systems to centralized online platforms. The Event Booking Application is developed to address the need for a reliable, scalable, and intuitive system that bridges the gap between event organizers and attendees. By leveraging modern full-stack web technologies, this platform facilitates seamless event discovery, secure user registration, and hassle-free booking processes.

1.2 Aim of the Project

The primary aim of this project is to design and develop a robust web-based application that allows users to easily browse and book tickets for various departmental and institutional events.

1.3 Scope of the Project

The scope of the project encompasses:

- A user-facing frontend for browsing events, registering accounts, and booking tickets.
- An administrative dashboard for comprehensive CRUD operations on events and user bookings.
- An automated email notification system to send booking confirmations to users.
- A secure backend API to handle business logic and database interactions.
- A relational database schema to efficiently store user credentials, event details, and booking transactions.

1.4 Framework

The application is built using a modern JavaScript-based stack tailored for relational data:

- **Frontend:** React.js, React Router DOM, Axios, and Framer Motion.
- **Backend:** Node.js and Express.js.

- **Database:** MySQL relational database/h2 console

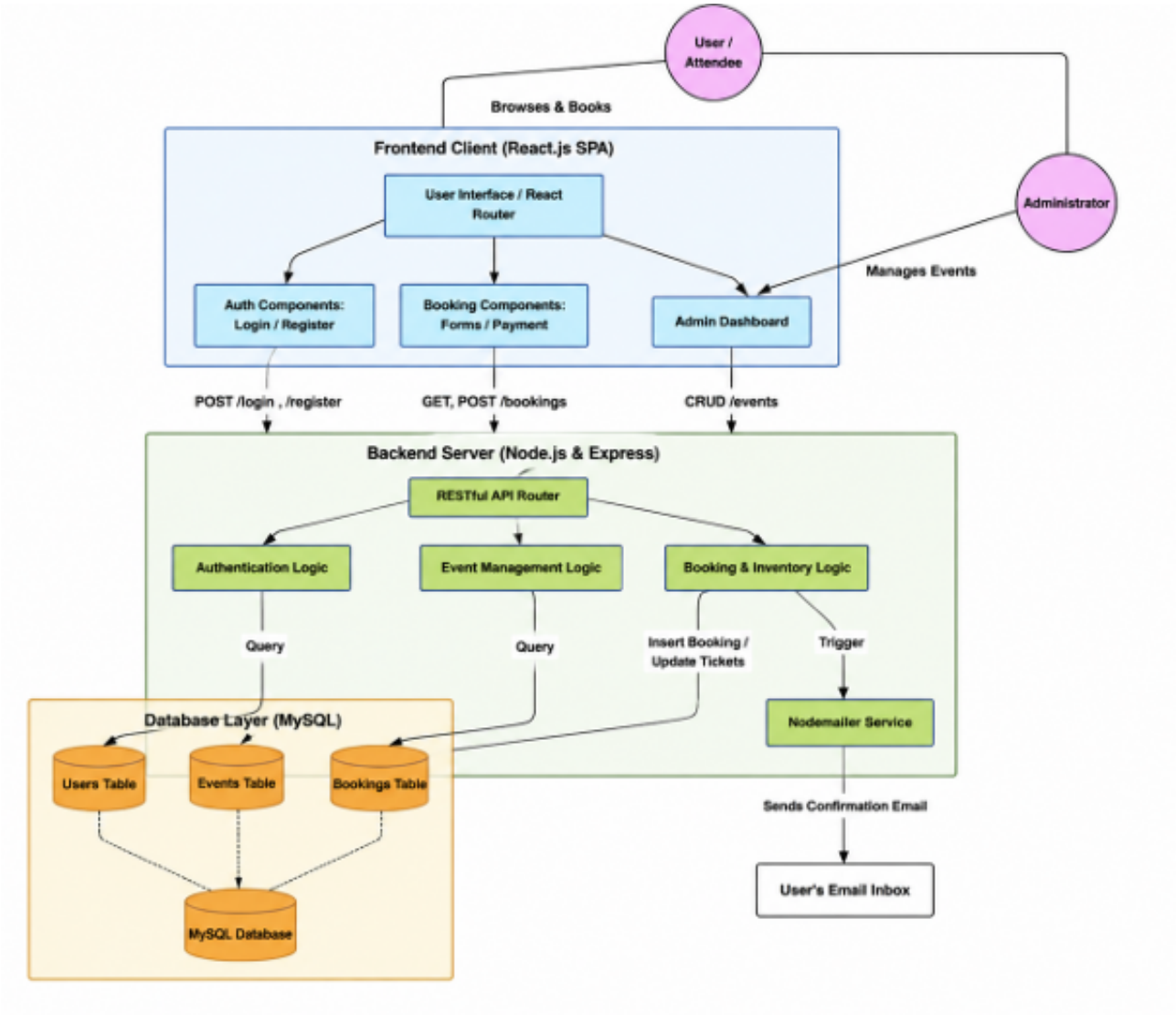

Figure 1.1: System Overview Diagram

Figure 1.1 shows the architecture of the Smart Campus Event Management System. It includes a React.js frontend, a Node.js backend, and a MySQL database. Users can browse events and book tickets, while the admin manages events. The system ensures scalability and smooth communication between components.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

The market for event booking platforms is highly competitive, featuring industry leaders such as Eventbrite, Meetup, and BookMyShow.

- **Eventbrite:** Offers extensive tools for organizers but can be highly complex and introduces significant transaction fees, making it less ideal for localized or institutional events.
- **Meetup:** Focuses primarily on community-building and recurring group meetups. It lacks the streamlined ticketing and pricing structure needed for standalone commercial or departmental events.
- **BookMyShow:** A dominant player in the entertainment ticketing sector, but its architecture is heavily tailored for large-scale cinema and concert events, making it overkill for smaller custom event

management.

The proposed Event Booking Application differentiates itself by offering a lightweight, highly customizable, and cost-effective solution specifically tailored for institutional, departmental, or mid-sized community events without the overhead of enterprise-level platforms. Additionally, the proposed system provides a simplified user interface and streamlined workflow that ensures ease of use for both administrators and users. Unlike existing platforms that may overwhelm users with complex features, this application focuses on essential functionalities such as event creation, booking, and confirmation, making it more accessible for academic and small-scale organizational use.

Furthermore, the application is designed with flexibility and scalability in mind, allowing future enhancements such as secure authentication, real-time payment integration, and advanced filtering options. This makes the system adaptable to evolving user needs while maintaining a cost-effective and efficient solution compared to large commercial platforms.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend environment is initialized using Create React App (CRA). Key dependencies include ‘react-router-dom‘ for client-side routing, ‘axios‘ for HTTP API requests, ‘framer-motion‘ for UI animations, and ‘react-toastify‘ for user feedback notifications. The development environment requires Node.js and npm to be installed locally.

3.1.2 Structure of the Project

The frontend source code (located in the ‘src‘ directory) is highly modularized:

- **App.js**: The root component handling global state (user/admin login status) and application routing.
- **components/**: Contains reusable UI components such as Navbar, Header, BookingForm, Login, UserLogin, UserRegister, Payment,

and Confirmation.

- **index.js**: The entry point that renders the React application into the browser DOM.

3.1.3 Execution Procedure

To execute the frontend: 1. Navigate to the project root directory in the terminal. 2. Execute `npm install` to install all necessary dependencies. 3. Execute `npm start` to launch the development server. The application will be accessible at `http://localhost:3000`.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend is built using Node.js. The environment is initialized with `npm init` and relies on several critical packages: `express` for setting up the web server and routing, `mysql2` for database connectivity, `cors` for Cross-Origin Resource Sharing, `dotenv` for managing sensitive environment variables, and `nodemailer` for automated email services.

3.2.2 Structure of the Project

The backend architecture is centralized in the `backend/server.js` file, which includes:

3.2.1 3.2.2 Structure of the Project

The backend architecture is centralized in the `backend/server.js` file, which includes:

- Server configuration and middleware setup (`cors`, `express.json`) for handling HTTP requests and JSON data.
- MySQL database connection initialization with automatic table creation for users, events, and bookings.
- RESTful API routes categorized into Users (`/register`, `/login`), Events (`/events`), and Bookings (`/bookings`).
- Modular route handling to improve maintainability and scalability of the application.
- Integration of external services such as Nodemailer for sending booking confirmation emails.

This structured design ensures efficient data handling, clear organization of functionalities, and smooth communication between the client and server components.

3.2.3 Execution Procedure

To execute the backend: 1. Navigate to the `backend` directory. 2. Execute `npm install` to install backend dependencies. 3. Ensure a MySQL server is running locally with the credentials specified in the configuration. 4. Execute `node server.js` to start the backend server, which will run on `http://localhost:5001`.

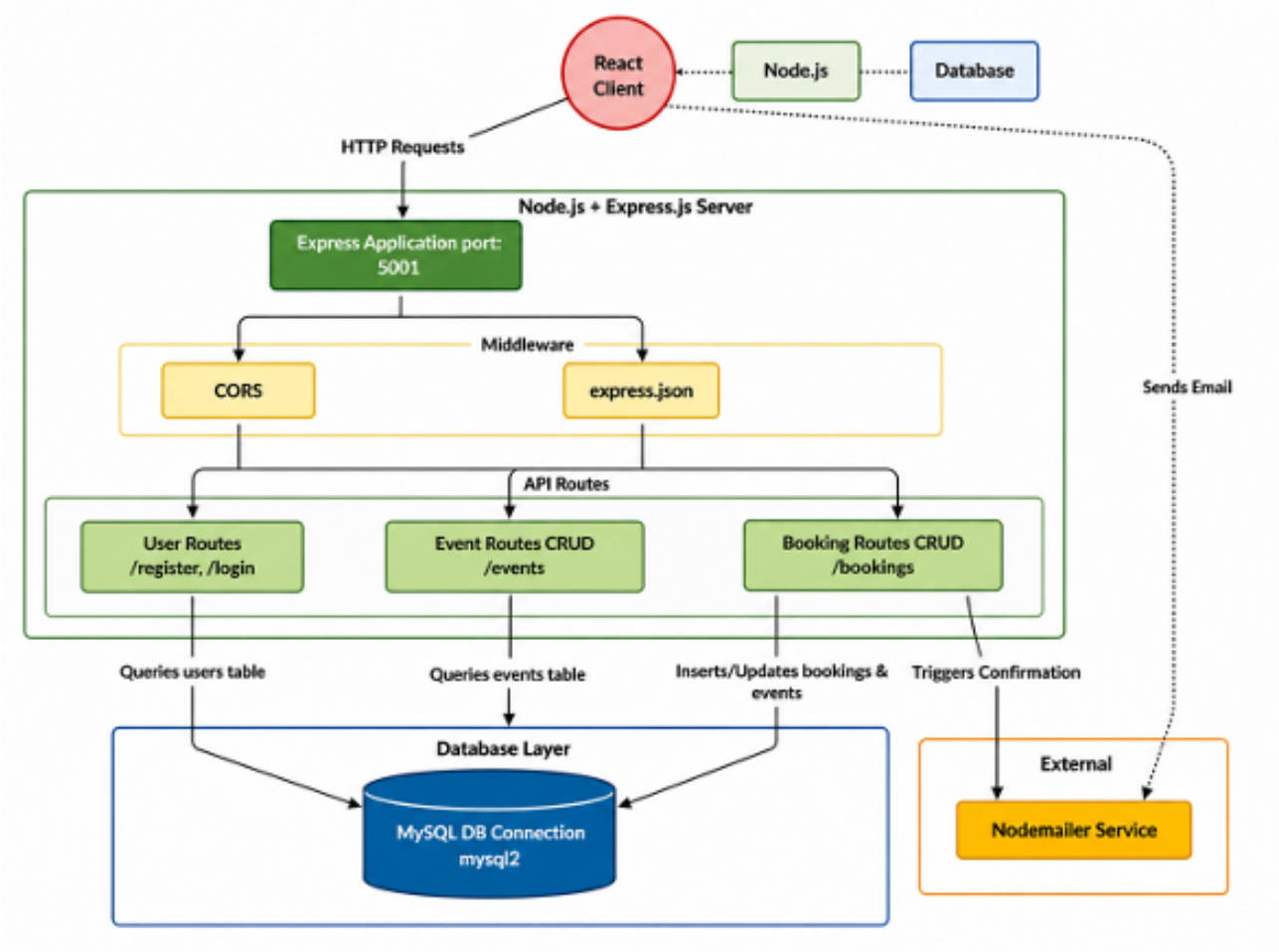

Figure 3.1: Backend Architecture

Figure 3.1 shows the backend architecture of the system. It includes a Node.js and Express server handling API requests, middleware, and route management. The system interacts with a MySQL database and

uses Nodemailer for sending confirmation emails.

3.3 Database Implementation

3.3.1 Database Configuration

The application utilizes MySQL for persistent data storage. The database is named `eventdb`. The connection is established securely using the `mysql2` library, connecting to `localhost` on the default MySQL port utilizing root credentials.

3.3.2 Schema Structure

The schema is relationally designed to ensure data integrity and prevent redundancy across user profiles, available events, and transactional booking records.

3.3.3 Tables Created

- **users**: Stores user credentials. Columns include `id` (Primary Key, Auto Increment), `name`, `email` (Unique), and `password`.
- **events**: Stores event details. Columns include `id` (Primary Key), `name`, `department`, `date`, `venue`, `price`, and `tickets` (representing available capacity).
- **bookings**: Stores transaction records. Columns include `id` (Primary Key), `name`, `email`, `department`, `tickets` (quantity booked), and `event_id` (linking to the events table).

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The project follows a standard Client-Server architecture. The React frontend acts as the client, rendering the user interface as a Single Page Application (SPA). It communicates asynchronously via Axios HTTP requests to the Node.js/Express backend. The backend serves as the intermediary business logic layer, processing requests, interacting with the MySQL database via SQL queries, and returning JSON responses back to the client.

4.2 DataFlow Diagram

1. The User interacts with the React UI to submit a booking form.
2. The React component dispatches an Axios POST request to the backend API (`/bookings`).
3. The Express server receives the request, validates the data pay-

load, and executes a MySQL `INSERT` query to permanently record the booking.

4. The server executes an `UPDATE` query to accurately decrement the available ticket count in the `events` table.

5. The server utilizes Nodemailer to dispatch an automated confirmation email to the user's provided email address.

6. A success response is returned to the client, triggering a UI state update to display a confirmation message via React Toastify.

4.3 Module 1: User Authentication & Registration

This module handles the creation of new user accounts and the authentication of existing users. It features form validation on the frontend and secure credential verification against the MySQL `users` table on the backend, accurately managing login states within the React application to protect private routes.

4.4 Module 2: Event Management (Admin Dashboard)

This module provides administrators with a protected interface to perform comprehensive CRUD operations on the event catalog. Administrators can create new events by specifying details like name, date, venue, price, and ticket capacity, as well as update existing event

details or remove canceled events.

4.5 Module 3: Booking & Notification System

This module enables authenticated users to select events and reserve tickets. It ensures transactional integrity by recording the booking, updating the available ticket inventory in real-time within the database, and automatically triggering a booking confirmation email using an SMTP service (Nodemailer).

Advantages of this Project

- **Scalability:** The decoupled React frontend and Express backend can be scaled and maintained independently.
- **Real-time Inventory Control:** Ticket availability is updated synchronously upon booking, effectively preventing double-booking or over-booking.
- **Automated Communication:** Integrated email notifications enhance the user experience by providing immediate, reliable booking confirmations.

Disadvantages

- **Security Constraints:** Passwords are currently managed in plain

text; implementing bcrypt hashing is required for production-level security.

- **Payment Integration:** The payment process is visually simulated and currently lacks integration with a live third-party payment gateway (e.g., Stripe, Razorpay) for actual financial processing.

This project also promotes modular development by clearly separating frontend, backend, and database layers, which improves maintainability and future scalability. The user-friendly interface ensures smooth navigation, making it easy for users to browse events and complete bookings efficiently. Additionally, the system supports real-time interaction between components, enhancing responsiveness and overall performance.

Another limitation of the system is that it is currently deployed only in a local environment, which restricts accessibility to external users. The system also lacks advanced features such as role-based access control and detailed analytics for administrators. Furthermore, error handling and validation mechanisms can be enhanced to improve system robustness and reliability in real-world scenarios.

Chapter 5

RESULTS AND DISCUSSIONS

The Event Booking Application successfully fulfills its primary functional requirements. Users can intuitively navigate the platform, register accounts securely, and book event tickets without friction. Administrators have full, reliable control over event listings through the dedicated dashboard.

The integration of React Router provides a seamless Single Page Application experience, resulting in fast navigation without full page reloads. The Express backend efficiently processes concurrent API requests, and the MySQL database accurately maintains data consistency, particularly during ticket quantity deductions. During local testing, the Nodemailer integration reliably delivered email confirmations immediately upon successful bookings.

Feature	**Eventbrite**	**BookMyShow**	**Proposed App**
Custom Branding	Limited	Limited	High
Transaction Fees	High	Moderate	None (Self-Hosted)
Setup Complexity	High	Moderate	Low
Target Audience	Commercial Events	Entertainment Sector	Institutional / Departmental
Data Ownership	Third-Party	Third-Party	Fully Owned
Flexibility	Limited Customization	Limited Customization	Highly Customizable
Cost Efficiency	Low	Moderate	High

Table 5.1: Application Feature Comparison

Table 5.1 presents a comparison of the proposed application with existing platforms such as Eventbrite and BookMyShow. The proposed system offers advantages like no transaction fees, low setup complexity, full data ownership, and high customization, making it more suitable for institutional and departmental use.

5.1 System Screenshots and Results

The developed Event Booking Application is currently deployed in a local environment (localhost) and includes the following modules:

5.1.1 User Login Page

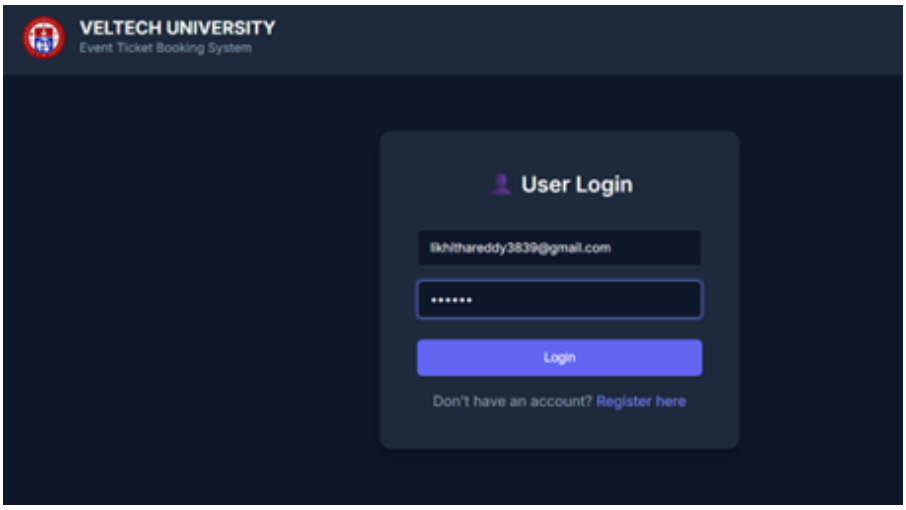

Figure 5.1: User Login Page

Figure 5.1 shows the login interface where registered users securely access the system using their credentials. It ensures proper authentication before granting access to booking features.

5.1.2 User Registration Page

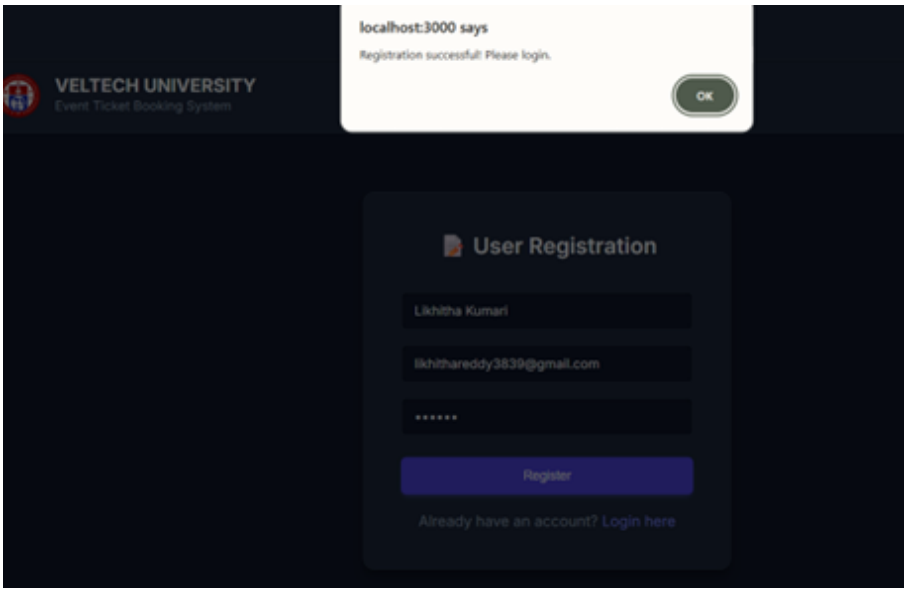

Figure 5.2: User Registration Page

Figure 5.2 shows the registration page where new users can create an account by entering details such as name, email, and password. The data is stored for future authentication.

5.1.3 Admin Dashboard

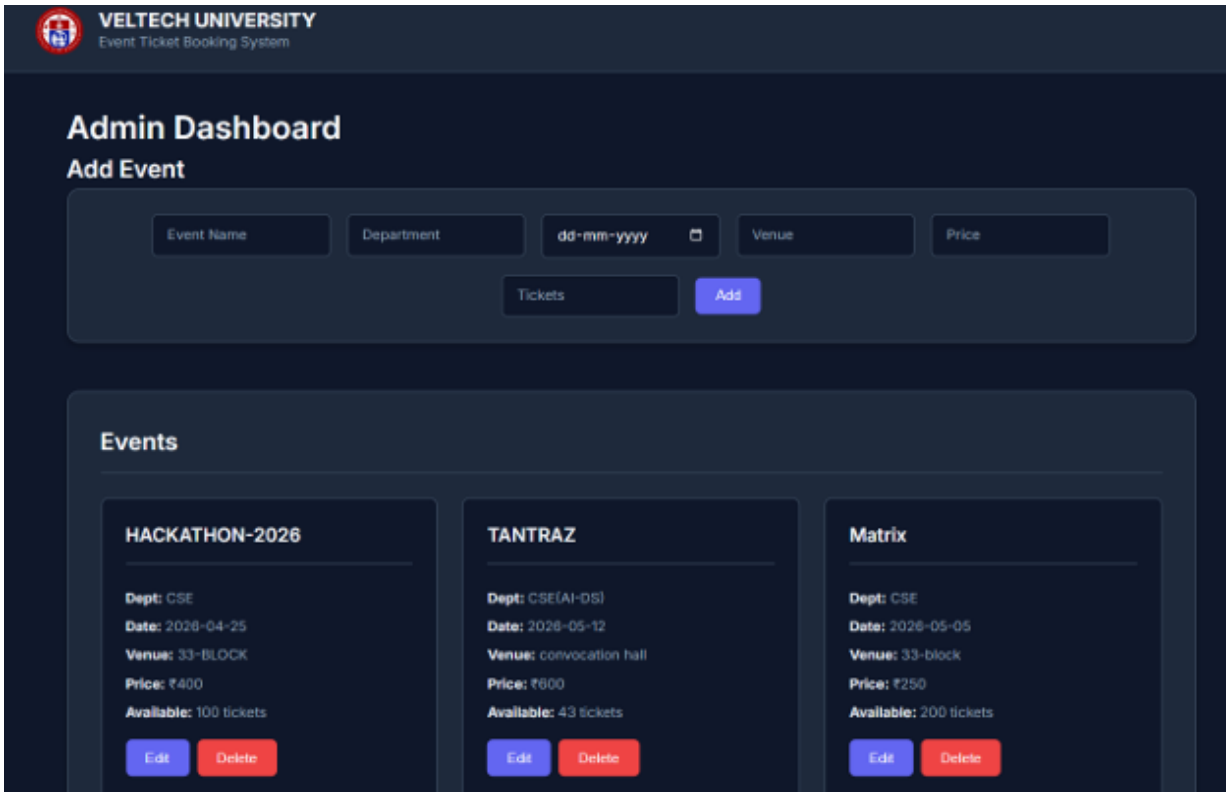

Figure 5.3: Admin Dashboard with Event Management

Figure 5.3 shows the admin dashboard used to manage events. It allows adding, editing, and deleting events, while displaying details like date, venue, price, and ticket availability in a structured format.

5.1.4 Event Booking Page

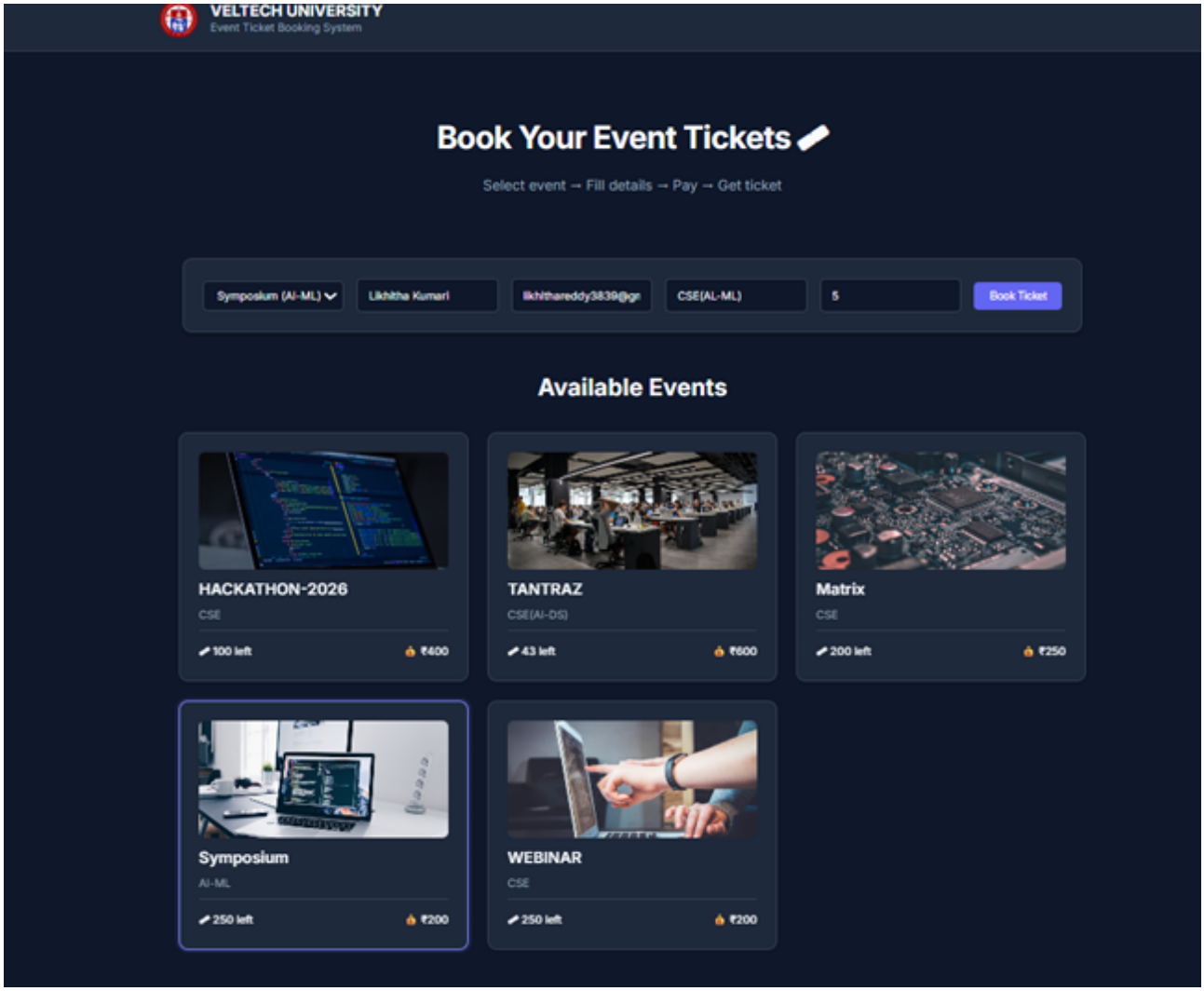

Figure 5.4: Event Booking Interface

Figure 5.4 shows the booking interface where users can view events and select tickets. It provides real-time updates on availability and event details.

5.1.5 Payment Page

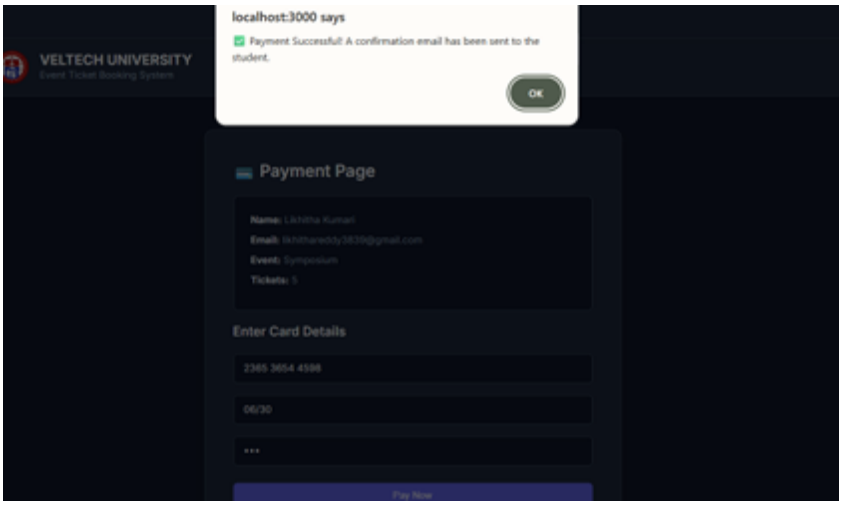

Figure 5.5: Payment Page

Figure 5.5 shows the payment page where users enter details to complete the booking process securely.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Event Booking Application demonstrates the effectiveness of utilizing modern web technologies to build a robust and interactive platform. By integrating React.js, Node.js, Express, and MySQL, the system provides a seamless experience for users, from browsing events to booking tickets and receiving confirmation emails.

The application successfully implements key functionalities such as user authentication, event management, ticket booking, payment simulation, and email notifications. The system is efficient, user-friendly, and suitable for institutional event management.

6.2 Future Enhancements

- **Secure Authentication:** Implement JSON Web Tokens (JWT) and bcrypt to enhance security and ensure safe login mechanisms.
- **Payment Gateway Integration:** Integrate real-time payment services such as Stripe or PayPal for secure transactions.
- **Advanced Filtering:** Enable users to search, sort, and filter events based on categories, date, and keywords.
- **Digital Ticketing:** Generate QR codes for each booking and include them in confirmation emails for easy verification.
- **Cloud Deployment:** Deploy the system using cloud platforms such as AWS, Vercel, or Netlify.

6.3 SOURCE CODE IMPLEMENTATION

6.3.1 Backend Server and API Logic (server.js)

This section contains the core backend logic, including the MySQL database connection, RESTful API routes for events and bookings, and the Nodemailer integration for automated email confirmations.

```
const express = require("express");
const mysql = require("mysql2");
const nodemailer = require("nodemailer");
const app = express();
app.use(express.json());
// Database Connection
const db = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "your_password",
  database: "eventdb"
});
// API Route: Handle Bookings & Email Notification
app.post("/bookings", (req, res) => {
  const { name, email, tickets, eventId } = req.body;
  const sql = "INSERT INTO bookings (name, email, tickets, event_id) VALUES (?, ?, ?, ?)";
  db.query(sql, [name, email, tickets, eventId], (err) => {
    if (err) return res.json(err);

    // Reduce ticket count in events table
    db.query("UPDATE events SET tickets = tickets - ? WHERE id = ?", [tickets, eventId]);

    // Send Confirmation Email
    const mailOptions = {
      from: process.env.EMAIL_USER,
      to: email,
      subject: "Booking Confirmation",
      text: `Hello ${name}, your booking is confirmed!`
    };
    transporter.sendMail(mailOptions);
    res.json({ message: "Booking successful" });
  });
});

app.listen(5001, () => console.log("Server running on port 5001"));
```

Listing 6.1: server.js - Backend Implementation

6.3.2 Frontend Booking Form (BookingForm.js)

The frontend implementation handles the user interface for selecting events and submitting booking details using React hooks and Axios for API communication.

```
import React, { useEffect, useState } from "react";
import axios from "axios";

function BookingForm() {
  const [events, setEvents] = useState([]);
  const [form, setForm] = useState({ name: "", email: "", tickets: "" });

  useEffect(() => {
    axios.get("http://localhost:5001/events")
      .then(res => setEvents(res.data));
  }, []);

  const handleSubmit = () => {
    axios.post("http://localhost:5001/bookings", form)
      .then(res => alert("Booking Successful!"));
  };

  return (
    <div className="form-box">
      <input name="name" placeholder="Name" onChange={(e) => setForm({...form,
          name: e.target.value})} />
      <input name="email" placeholder="Email" onChange={(e) => setForm({...form,
           email: e.target.value})} />
      <button onClick={handleSubmit}>Book Ticket</button>
    </div>
  );
}
```

Listing 6.2: BookingForm.js - Frontend Logic

6.3.3 Database Schema Implementation (SQL)

The following SQL queries define the structure of the relational database used to store event data and user transactions.

```sql
CREATE TABLE events (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  department VARCHAR(255),
  price DECIMAL(10,2),
  tickets INT
);

CREATE TABLE bookings (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(255),
  email VARCHAR(255),
  tickets INT,
  event_id INT,
  FOREIGN KEY (event_id) REFERENCES events(id)
);
```

Listing 6.3: Database Schema Structure

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

Level	Attribute	Characteristics	WKs	Justification
WP1	Depth of Knowledge	Requires in-depth engineering	WK4, WK5	Knowledge of React JS, hooks, validation, UI design
WP2	Conflicting Requirements	Technical and non-technical constraints	WK4, WK5	Balances usability with booking constraints and validation
WP3	Depth of Analysis	Analytical and design thinking	WK3, WK4	Component design, state handling, dynamic updates
WP4	Familiarity	Partially new problems	WK4	Real-time booking and ticket updates
WP5	Applicable Codes	Limited standard solutions	WK6	Custom booking and validation logic
WP6	Stakeholder Needs	Multiple stakeholders	WK5, WK6	Students, faculty, organizers
WP7	Interdependence	System-level integration	WK3, WK5	UI + state + validation integration

Table 7.1: Mapping of CEP Attributes

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Use of technologies	React JS, HTML, CSS
EA2	Level of Interactions	Complex module interaction	Event display + booking + validation
EA3	Innovation	Modern technologies	Hooks, conditional rendering
EA4	Consequences	Impact on users	Simplifies booking
EA5	Familiarity	Beyond standard	Dynamic real-time system

Table 7.2: Mapping of CEA Attributes

7.3 SDG Mapping (CEP Section)

SDG No.	SDG Title	Relevance
SDG 4	Quality Education	Supports academic events and learning platforms
SDG 9	Innovation	Promotes use of modern React-based systems
SDG 10	Reduced Inequalities	Accessible system for all types of users
SDG 11	Sustainable Communities	Encourages organized and efficient event management

Table 7.3: SDG Mapping

REFERENCES

[1] Eventbrite Platform. An online event management and ticketing platform that allows users to create, manage, and promote events efficiently. Available at: https://www.eventbrite.com

[2] BookMyShow Platform. A popular entertainment and event booking platform used for booking tickets for movies, concerts, and live events. Available at: https://www.bookmyshow.com

[3] React Documentation. Official documentation for building interactive user interfaces using React.js library. Available at: https://react.dev

[4] React Router Documentation. Provides routing solutions for React applications to enable navigation between different components. Available at: https://reactrouter.com

[5] Node.js Documentation. Official documentation for Node.js, used for building scalable server-side applications. Available at: https://nodejs.org

[6] Express.js Documentation. A fast and minimal web framework for Node.js used to build RESTful APIs. Available at: https://expressjs.com

[7] MySQL Documentation. Official reference for MySQL database, covering database design, queries, and management. Available at: https://dev.mysql.com/doc/

[8] MySQL2 Node.js Driver. A Node.js library used to connect and interact with MySQL databases efficiently. Available at: https://www.npmjs.com/package/mysql2

[9] Nodemailer Documentation. A module used in Node.js applications to send emails such as booking confirmations. Available at: https://nodemailer.com

[10] Axios HTTP Client. A promise-based HTTP client used for making API requests from the frontend. Available at: https://axios-http.com